

Three vertical lines of varying heights and colors (yellow, orange, and blue) are positioned on the left side of the slide.

3. Evaluación del *software*

Análisis estático

Miguel Ángel Latre
Javier Noguerras Iso

Temario (técnicas y herramientas)

	Técnicas	Herramientas
Técnicas generales basadas en la estructura	Puntos de decisión Camino con profundidad	Medición de cobertura
Técnicas generales basadas en la especificación	Particiones equivalencia Pruebas de pares / tablas ortogonales Análisis de valores límite	Pict 3.3 AllPairs
Pruebas de desarrollo	Generales	Aserciones; JUnit 4; JUnit 5/6
de unidad		
de integración	Sustitutos	Mockito
Pruebas de sistema y aceptación (con GUI)	Generales	Selenium Espresso, UI Automator Cucumber
Evaluación (documentación)	Inspecciones Listas de comprobación	
Análisis estático (código)	Patrones de errores Verificación formal	Inspector de IntelliJ/Android Studio JML

Índice

□ Técnicas

❖ Patrones de errores

- Análisis sintáctico y semántico del compilador
- Referencias nulas
- Excepciones no capturadas
- Uso de memoria y recursos

❖ Verificación formal

□ Herramientas para el análisis estático de código:

- ❖ Inspector de IntelliJ/Android Studio, SONAR, ...
- ❖ ESC/Java

Definición

❏ Análisis estático del código

- ❖ Proceso de evaluar el código sin ejecutarlo

- ❖ Objetivos

- Encontrar y corregir defectos
- Mejorar el código
 - Facilitar el desarrollo y el mantenimiento
- Aumentar la eficiencia del código
- Aumentar la seguridad del código

Técnicas

❏ Análisis estático del código

❖ Patrones de errores

- Análisis sintáctico y semántico del compilador
- Referencias nulas
- Excepciones no capturadas
- Uso de memoria y recursos
- ...

❖ Verificación formal

- Prueba formal y matemática de que un programa cumple con sus especificaciones
 - Centrado habitualmente en requisitos funcionales
- Habitualmente costoso
- Visto en Programación 2

Análisis sintáctico y semántico del compilador

❑ Errores sintácticos

- ❖ El programa no se puede compilar o ejecutar.
- ❖ El programa se podría compilar o ejecutar, pero las reglas sintácticas/semánticas del lenguaje impiden su compilación/ejecución.

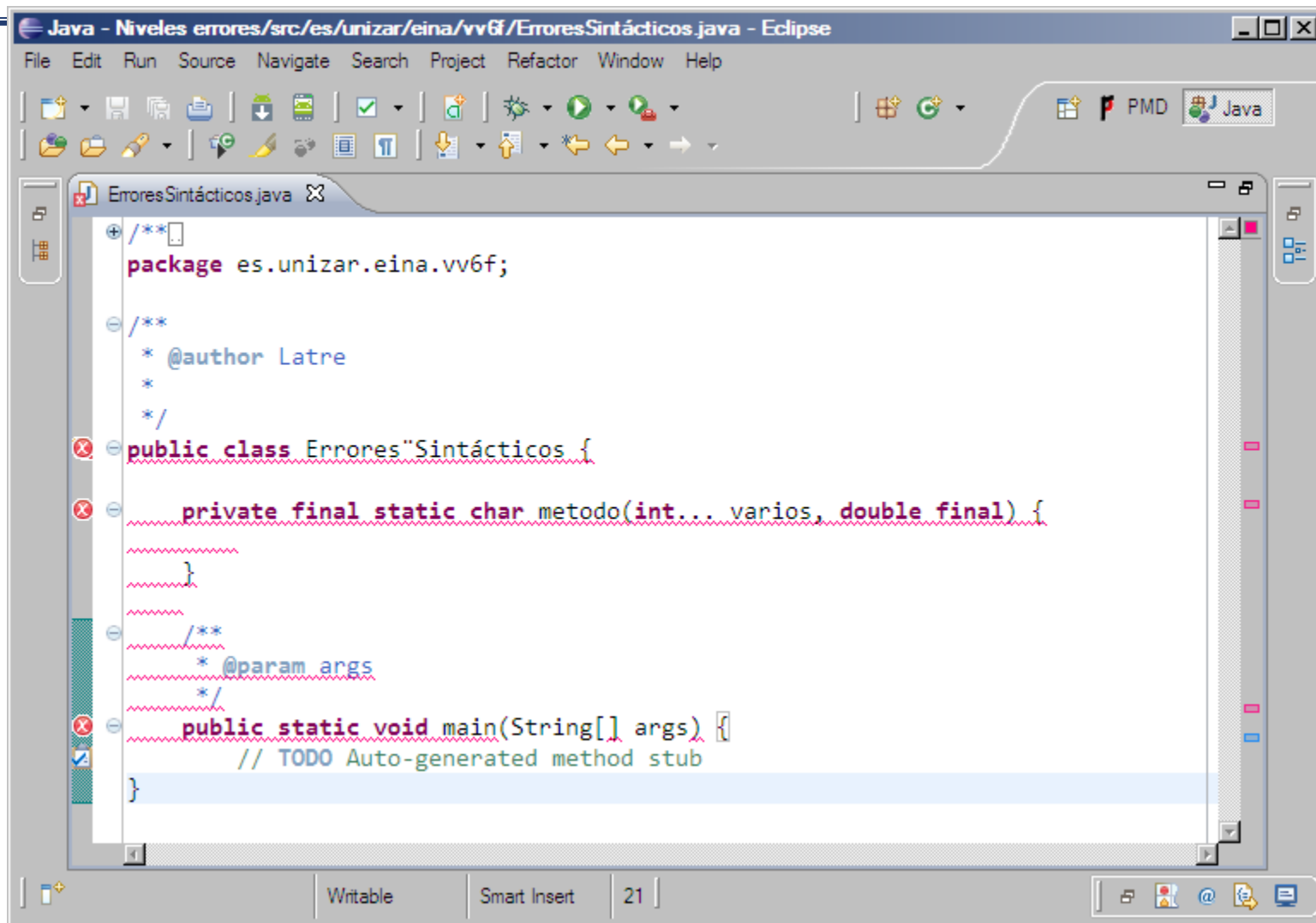
❑ Advertencias

- ❖ Construcciones válidas según el lenguaje de programación, pero son defectos potenciales

❑ Analizadores estáticos

- ❖ Buscan con más profundidad construcciones válidas pero potencialmente defectuosas.
- ❖ Permiten reestructurar o mejorar el código.

Errores sintácticos



The screenshot shows the Eclipse IDE interface. The title bar reads "Java - Niveles errores/src/es/unizar/eina/vv6f/ErroresSintácticos.java - Eclipse". The menu bar includes File, Edit, Run, Source, Navigate, Search, Project, Refactor, Window, and Help. The toolbar contains various icons for file operations, running, and debugging. The editor window displays the following Java code:

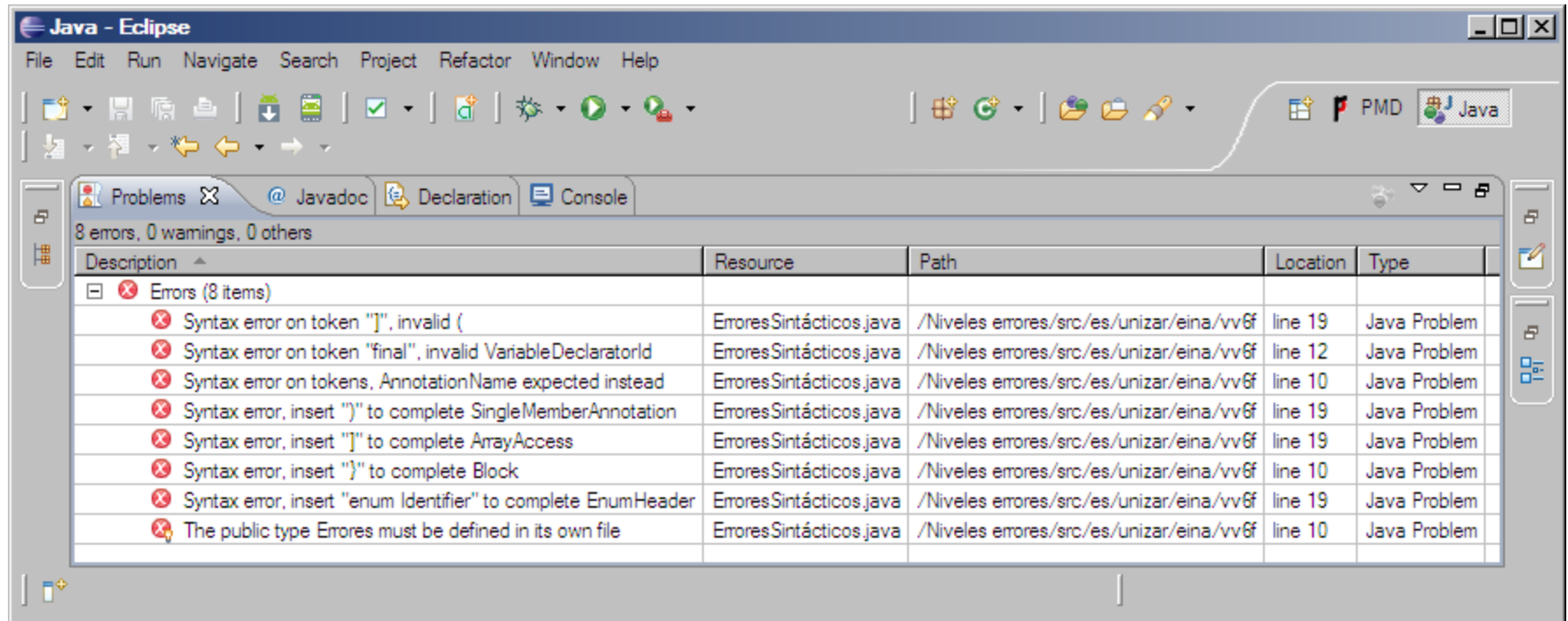
```
/**  
package es.unizar.eina.vv6f;  
  
/**  
 * @author Latre  
 *  
 */  
public class Errores"Sintácticos {  
  
    private final static char metodo(int... varios, double final) {  
  
    }  
  
    /**  
     * @param args  
     */  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
    }  
}
```

Four syntax errors are indicated by red squiggly lines and error icons in the left margin:

- Line 10: `public class Errores"Sintácticos {` - Invalid character `"` in class name.
- Line 14: `private final static char metodo(int... varios, double final) {` - Invalid parameter name `final`.
- Line 20: `/**` - Unterminated comment.
- Line 24: `public static void main(String[] args) {` - Unterminated comment.

The status bar at the bottom shows "Writable", "Smart Insert", and the line number "21".

Errores sintácticos



The screenshot shows the Eclipse IDE interface. The top menu bar includes File, Edit, Run, Navigate, Search, Project, Refactor, Window, and Help. Below the menu is a toolbar with various icons. The main workspace area is empty. On the left side, the 'Problems' view is open, showing a list of 8 errors. The errors are listed in a table with columns: Description, Resource, Path, Location, and Type. The errors are all 'Java Problem' type and are located in the file 'ErroresSintácticos.java' at various line numbers (10, 12, 19). The errors include syntax errors on tokens, invalid variable declarator, annotation name expected instead, and missing closing braces for annotations, array access, and blocks. The last error states that the public type 'Errores' must be defined in its own file.

Description	Resource	Path	Location	Type
Errors (8 items)				
Syntax error on token "[", invalid (	ErroresSintácticos.java	/Niveles errores/src/es/unizar/eina/vv6f	line 19	Java Problem
Syntax error on token "final", invalid VariableDeclaratorId	ErroresSintácticos.java	/Niveles errores/src/es/unizar/eina/vv6f	line 12	Java Problem
Syntax error on tokens, AnnotationName expected instead	ErroresSintácticos.java	/Niveles errores/src/es/unizar/eina/vv6f	line 10	Java Problem
Syntax error, insert ")" to complete SingleMemberAnnotation	ErroresSintácticos.java	/Niveles errores/src/es/unizar/eina/vv6f	line 19	Java Problem
Syntax error, insert "]" to complete ArrayAccess	ErroresSintácticos.java	/Niveles errores/src/es/unizar/eina/vv6f	line 19	Java Problem
Syntax error, insert "}" to complete Block	ErroresSintácticos.java	/Niveles errores/src/es/unizar/eina/vv6f	line 10	Java Problem
Syntax error, insert "enum Identifier" to complete EnumHeader	ErroresSintácticos.java	/Niveles errores/src/es/unizar/eina/vv6f	line 19	Java Problem
The public type Errores must be defined in its own file	ErroresSintácticos.java	/Niveles errores/src/es/unizar/eina/vv6f	line 10	Java Problem

Java - Niveles errores/src/es/unizar/eina/vv6f/ErroresSintácticos.java - Eclipse

File Edit Run Source Navigate Search Project Refactor Window Help

Package Explorer Outline

```
package es.unizar.eina.vv6f;

import java.io.InputStream;
import java.util.InputMismatchException;
import java.util.Scanner;

public class ErroresSintácticos {

    public static void main(String[] argumentos) {
        InputStream s = new Scanner(System.in);
        try {
            while (true) {
                System.out.println(s.nextInt());
            }
            System.out.println("Fin");
        }
        catch (InputMismatchException ex) {
            // Salida del bucle
        }
    }
}
```

Outline

- es.unizar.eina.vv6f
 - import declarations
 - ErroresSintácticos
 - main(String[]) : void

Problems Javadoc Declaration Console

2 errors, 0 warnings, 0 others

Description	Resource	Path	Location
Errors (2 items)			
The method nextInt() is undefined for the type InputStream	ErroresSintácticos.java	/Niveles errores/src/es/unizar/eina/vv6f	line 13
Type mismatch: cannot convert from Scanner to InputStream	ErroresSintácticos.java	/Niveles errores/src/es/unizar/eina/vv6f	line 10

Writable Smart Insert 10 : 20

Advertencias

```
public static void main(String[] argumentos) {
    Scanner s = new Scanner(System.in);
    boolean seguir = true;
    int n = -1;
    try {
        n = s.nextInt();
    } catch (InputMismatchException ex) {
        seguir = false;
    }
    while (seguir = true) {
        System.out.println(n);
        try {
            n = s.nextInt();
        } catch (InputMismatchException ex) {
            seguir = false;
        }
    }
    System.out.println("Fin");
}
```

Advertencias

```
public static void main(String[] argumentos) {  
    Scanner s = new Scanner(System.in);  
    boolean seguir = true;  
    int n = -1;  
    try {  
        n = s.nextInt();  
    } catch (InputMismatchException ex) {  
        seguir = false;  
    }  
    while (seguir = true) {  
        System.out.println(n);  
        try {  
            n = s.nextInt();  
        } catch (InputMismatchException ex) {  
            seguir = false;  
        }  
    }  
    System.out.println("Fin");  
}
```

Fuga: el recurso «s» no se cierra

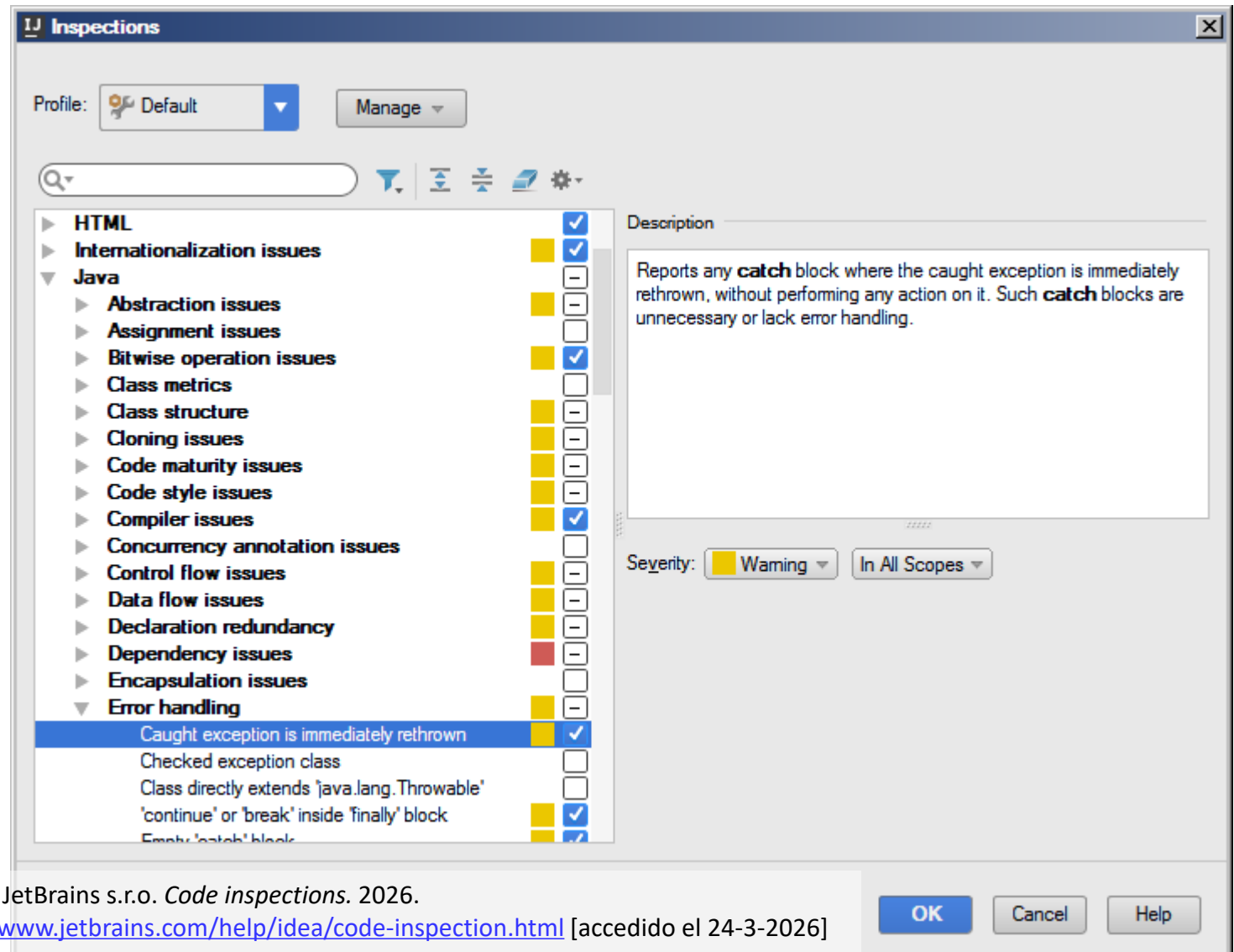
Nunca se accede al valor de la variable local «seguir»

El valor asignado a la variable local «seguir» no se usa nunca

Posible asignación accidental en lugar de una comparación

El valor asignado a la variable local «seguir» no se usa nunca

Advertencias en IntelliJ → *Inspections*



Advertencias en IntelliJ → *Inspections*

▼ Java		☐
▶ Abstraction issues	■	☐
▶ Assignment issues		☐
▶ Bitwise operation issues	■	☒
▶ Class metrics		☐
▶ Class structure	■	☐
▶ Cloning issues	■	☐
▶ Code maturity issues	■	☐
▶ Code style issues	■	☐
▶ Compiler issues	■	☒
▶ Concurrency annotation issues		☐
▶ Control flow issues	■	☐
▶ Data flow issues	■	☐
▶ Declaration redundancy	■	☐
▶ Dependency issues	■	☐
▶ Encapsulation issues		☐
▶ Error handling	■	☐
▶ Finalization issues	■	☐
▶ General		☐
▶ Imports	■	☐
▶ Inheritance issues	■	☐
▶ Initialization issues		☐
▶ Internationalization issues	■	☐
▶ J2ME issues	■	☐
▶ Java language level issues		☐
▶ Java language level migration aids		☐
▶ JavaBeans issues		☐
▶ Javadoc issues		☐
▶ JUnit issues	■	☐
▶ Logging issues		☐
▶ Memory issues		☐
▶ Method metrics		☐
▶ Modularization issues		☐
▶ Naming conventions	■	☐
▶ Numeric issues	■	☐
▶ Packaging issues		☐
▶ Performance issues	■	☐
▶ Portability issues		☐
▶ Probable bugs		☐
▶ Properties Files	■	☒
▶ Resource management issues		☐
▶ Security issues		☐
▶ Serialization issues	■	☐
▶ TestNG		☐
▶ Threading issues	■	☐
▶ toString() issues		☐
▶ Verbose or redundant code constructs	■	☒
▶ Visibility issues	■	☐

Patrones de errores

- Solo algunos:
 - ❖ Fuga de recursos
 - ❖ Análisis de punteros

Fuga de recursos

- ❑ Utilización de un recurso por parte de un programa que no es liberado posteriormente
- ❑ Tipos
 - ❖ Fugas de memoria
 - ❖ Fugas de descriptores (*handles*)

Gestión de memoria en Java

- ❑ Asignación de memoria con el operador **new**
- ❑ Recuperación de memoria por el recolector de basura
 - ❖ Invocado automáticamente por la máquina virtual
 - ❖ Invocado explícitamente a través de `System.gc()`
 - ❖ `Object.finalize()`
 - ❖ Anulación de referencias

Gestión de otros recursos

- ❑ Los objetos de clases que implementan los interfaces `java.io.Closeable` o `java.lang.AutoCloseable` se consideran recursos externos.
- ❑ Deben cerrarse utilizando el método `close()` cuando ya no sean necesarios.
- ❑ El inspector de IntelliJ puede analizar si el código que utiliza esas clases se adhiere a esa política.
- ❑ Ejemplo:
 - ❖

```
int length(File f) throws IOException {  
    InputStream stream = new FileInputStream(f);  
    return stream.available();  
}
```
 - ❖ *'FileInputStream' should be opened in front of a 'try' block and closed in the corresponding 'finally' block.*
- ❑ Instrucción *try-with-resources*
- ❑ Propiedad y responsabilidad de esos recursos

Punteros nulos

- El puntero nulo o la referencia nula fue diseñada por Tony Hoare en 1965 como parte del lenguaje Algol W.

I call it my billion-dollar mistake. It was the invention of the null reference in 1965. At that time, I was designing the first comprehensive type system for references in an object oriented language. My goal was to ensure that all use of references should be absolutely safe, with checking performed automatically by the compiler. But I couldn't resist the temptation to put in a null reference, simply because it was so easy to implement. This has led to innumerable errors, vulnerabilities, and system crashes, which have probably caused a billion dollars of pain and damage in the last forty years. In recent years, a number of program analysers like PREfix and PREfast in Microsoft have been used to check references, and give warnings if there is a risk they may be non-null. More recent programming languages like Spec# have introduced declarations for non-null references. This is the solution, which I rejected in 1965.

Punteros nulos

- El inspector de IntelliJ puede identificar automáticamente algunos casos:

```
Integer i = null;  
System.out.println(i.intValue());
```

Anotaciones para referencias en Java

❑ JSpecify

❖ <https://jspecify.dev/>

❑ Complementa el análisis estático que puede hacer el compilador con anotaciones hechas por el programador.

❑ Anotaciones:

❖ Para atributos, parámetros y tipos devueltos

➤ @NonNull: la referencia no puede ser nula.

➤ @Nullable: la referencia puede ser nula.

❖ Para módulos, paquetes, clases y métodos

➤ @NotNullMarked: indica que todas las referencias no anotadas del ámbito deben ser no nulas.

➤ @NullUnmarked: indica que todas las referencias no anotadas del ámbito pueden ser nulas.

Anotaciones para referencias en Java

```
static @Nullable String emptyToNull(@NonNull String str) {  
    if (str.isEmpty()) {  
        return null;  
    } else {  
        return str;  
    }  
}
```

```
static @NonNull String nullToEmpty(@Nullable String str) {  
    if (str == null) {  
        return "";  
    } else {  
        return str;  
    }  
}
```

Ejemplo

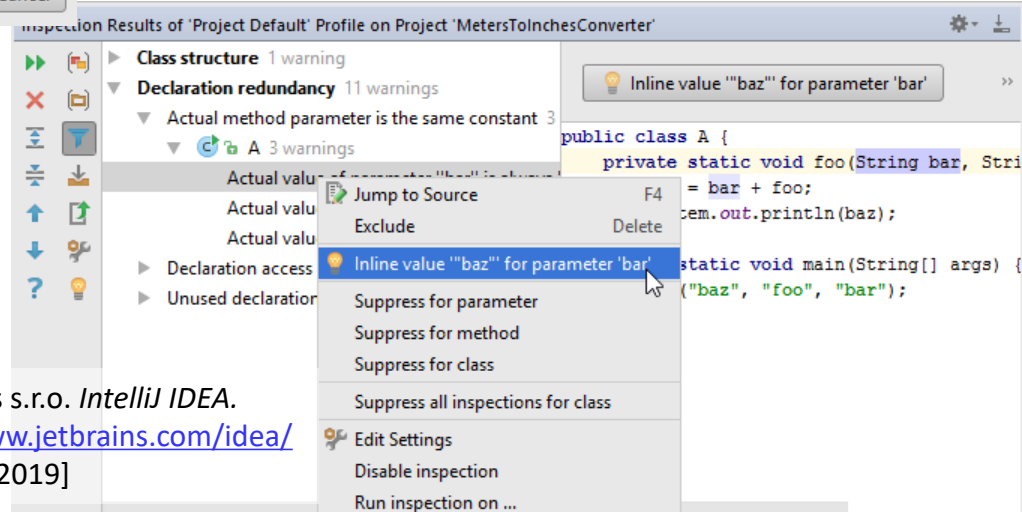
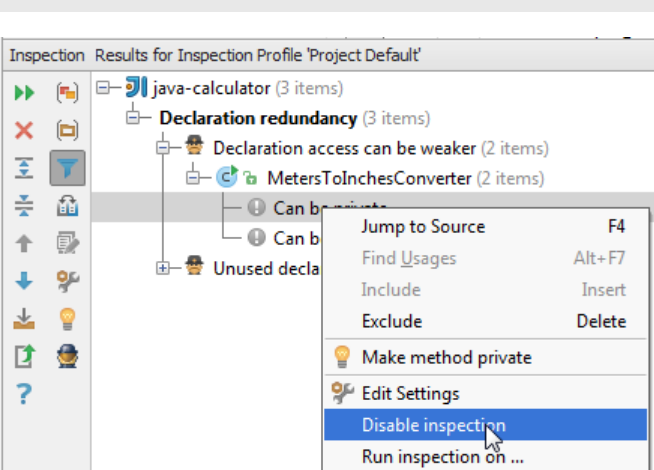
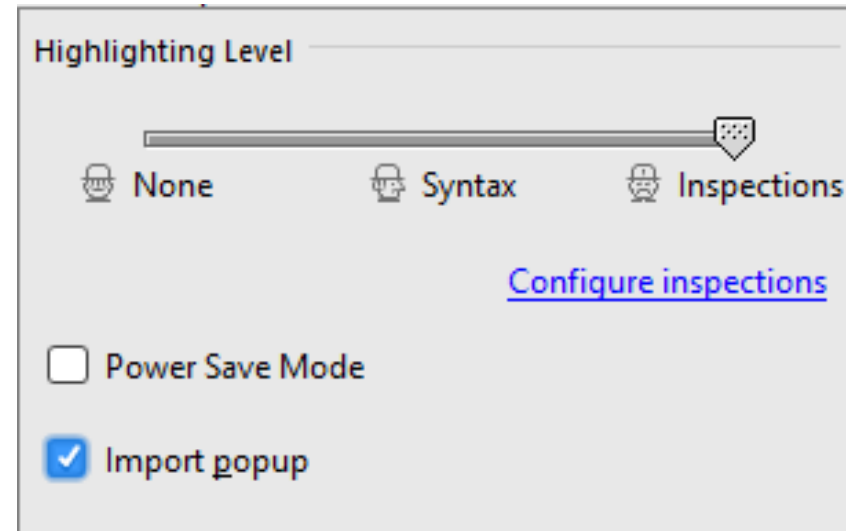
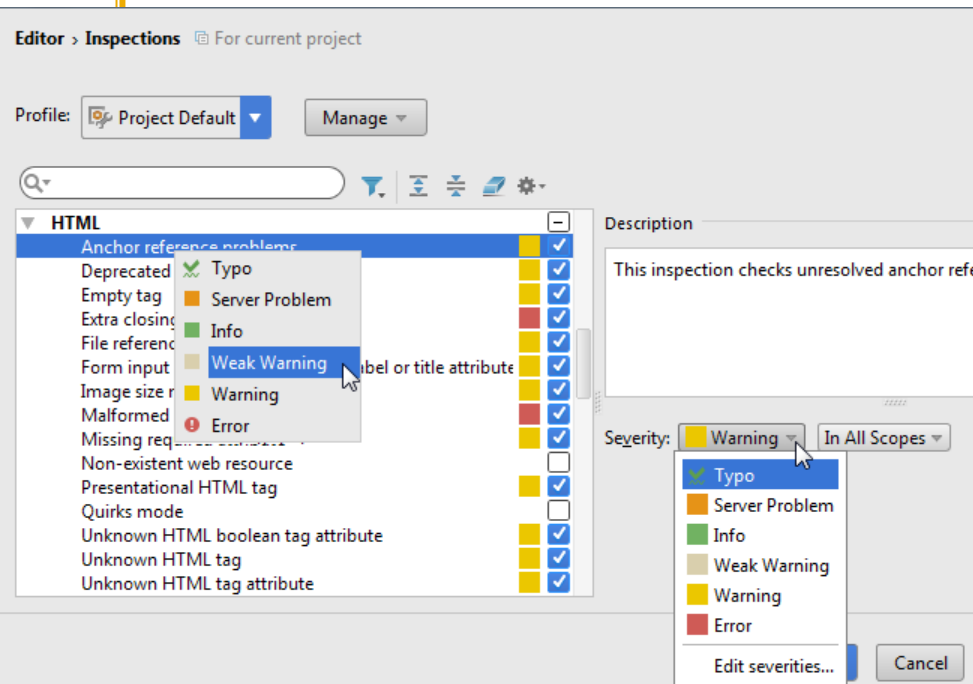
@Nullable

```
public record Venta(int producto, int cliente,
                    int cantidad, double precioUnitario) {
    public static @Nullable Venta leerVenta(
        DataInputStream f) throws IOException {
        try {
            int producto = f.readInt();
            int cliente = f.readInt();
            int cantidad = f.readInt();
            double precioUnitario = f.readDouble();
            return new Venta(producto, cliente, cantidad,
                              precioUnitario);
        } catch (EOFException ex) {
            return null;
        }
    }
}
```

Herramientas para análisis estático

- ❑ Inspector de IntelliJ
- ❑ SonarQube
- ❑ ...
- ❑ Métodos formales
 - ❖ JML
 - ❖ TLA⁺

Inspector de IntelliJ



Fuente: JetBrains s.r.o. IntelliJ IDEA.

2017. <https://www.jetbrains.com/idea/>
[accedido el 6-5-2019]

- ❑ *Continuous Inspection*: integrado con muchos servidores de integración continua
- ❑ Detección de defectos, malas prácticas y vulnerabilidades
- ❑ Soporte para más de 30 lenguajes
 - ❖ <https://www.sonarqube.org/>
 - ❖ <https://github.com/SonarSource/sonarqube>

app

←

→

↺

🏠

localhost:9000/dashboard?id=unizar-vv-notas

⋮

☆

🔍

Buscar

⬇

☆

⬆

☰

sonarqube

ProjectsIssuesRulesQuality ProfilesQuality Gates

?

Search for projects and files...

Log in

app

master

May 6, 2019, 4:21 PM

Version unspecified

OverviewIssuesSecurity ReportsMeasuresCodeActivity

Quality Gate

Passed

BugsVulnerabilities

New code: since previous version started 15 minutes ago

0A

0A

0A

0A

Bugs

Vulnerabilities

New Bugs

New Vulnerabilities

Code Smells

2dA

50

0A

0

Debt

Code Smells

New Debt

New Code Smells

Coverage

0.0%

0.0%

—

Coverage

Coverage on New Code

Duplications

0.0%

0

—

Duplications

Duplicated Blocks

Duplications on New Code

About This Project

No tags

XS

823

Lines of Code

Java823

Project Activity

May 6, 2019

unspecified

May 6, 2019

Project Analyzed

Show More

Quality Gate

(Default) Sonar way

Quality Profiles

(Java) Sonar way

Project Key

unizar-vv-notas

Copy

Get project badges

1474

39

Issues - app

← → ↺ 🏠

localhost:9000/project/issues?id=unizar-vv-notas&resolved=false

📄 ⋮ ☆ 🔍

Buscar

⬇️ ☆ ⬆️ ⬇️ ⋮

sonarqube

Projects Issues Rules Quality Profiles Quality Gates

🔍 Search for projects and files...

Log in

📁 app

🔗 master

May 6, 2019, 4:21 PM Version unspecified

Overview

Issues

Security Reports

Measures

Code

Activity

Filters

▼ Type

🐛 Bug 0

🔒 Vulnerability 0

💩 Code Smell 50

🔒 Security Hotspot 0

▼ Severity

🚫 Blocker 0

🔴 Critical 8

🔴 Major 8

🟢 Minor 14

🟡 Info 20

➤ Resolution

➤ Status

➤ Creation Date

➤ Language

➤ Rule

➤ Standard

➤ Tag

➤ Directory

➤ File

➤ Assignee

➤ Author

⬆️ ⬇️ to select issues

⬅️ ➡️ to navigate

🔄 1 / 50 issues

1d 7h effort

src/.../java/es/unizar/eina/notepadv3/NoteEdit.java

This class has 8 parents which is greater than 5 authorized. 2 months ago L24 design

💩 Code Smell 🔴 Major 🔵 Open Not assigned 5h30min effort

Remove this use of "startManagingCursor"; it is deprecated. 2 months ago L82 cert, cwe, obsolete

💩 Code Smell 🟢 Minor 🔵 Open Not assigned 15min effort

Move this method into "DatePickerFragment". 2 months ago L187 confusing

💩 Code Smell 🟢 Minor 🔵 Open Not assigned 5min effort

Remove this unused method parameter "v". 2 months ago L191 1 cert, misra, unused

💩 Code Smell 🔴 Major 🔵 Open Not assigned 5min effort

Remove this unused method parameter "v". 2 months ago L195 1 cert, misra, unused

💩 Code Smell 🔴 Major 🔵 Open Not assigned 5min effort

src/.../java/es/unizar/eina/notepadv3/Notepadv3.java

This class has 8 parents which is greater than 5 authorized. 2 months ago L24 design

💩 Code Smell 🔴 Major 🔵 Open Not assigned 5h30min effort

Remove this unnecessary cast to "ListView". 2 months ago L72 clumsy, redundant

💩 Code Smell 🟢 Minor 🔵 Open Not assigned 5min effort

Add a default case to this switch. 2 months ago L86 cert, cwe, misra

💩 Code Smell 🔴 Critical 🔵 Open Not assigned 5min effort

Remove this use of "startManagingCursor"; it is deprecated. 2 months ago L106 cert, cwe, obsolete

💩 Code Smell 🟢 Minor 🔵 Open Not assigned 15min effort

Remove this unused method p... X

← → ↺ 🏠 ⓘ localhost:9000/project/issues?id=unizar-vv-notas&open=AWqNgojmeJgvhNA_vrvu8 ... ☆ 🔍 Buscar ⬇️ ☆ >> ☰

sonarqube Projects Issues Rules Quality Profiles Quality Gates ? 🔍 Search for projects and files... Log in

app master May 6, 2019, 4:21 PM Version unspecified

Overview Issues Security Reports Measures Code Activity

← 4 / 50 issues ↺

NoteEdit.java alt + ↑ ↓ to navigate issue locations

src/.../eina/notepadv3/NoteEdit.java

This class has 8 parents which is greater than 5 authorized.
🔧 Code Smell 🚨 Major

Remove this use of "startManagingCursor"; it is deprecated.
🔧 Code Smell 🟢 Minor

Move this method into "DatePickerFragment".
🔧 Code Smell 🟢 Minor

Remove this unused method parameter "v".
🔧 Code Smell 🚨 Major +1

1 Remove this unused method parameter v".

Remove this unused method parameter "v".
🔧 Code Smell 🚨 Major +1

src/.../eina/notepadv3/Notepadv3.java

This class has 8 parents which is greater than 5 authorized.
🔧 Code Smell 🚨 Major

Remove this unnecessary cast to "ListView".
🔧 Code Smell 🟢 Minor

187 private Date getDate(int dataType) {

Move this method into "DatePickerFragment". ... 2 months ago ▾ L187 🔗
🔧 Code Smell 🟢 Minor 🔵 Open Not assigned 5min effort 🗑️ confusing

188 return dates[dataType];

189 }

190 }

191 public void showStartDatePickerDialog(View v) {

Remove this unused method parameter "v". ... 2 months ago ▾ L191 🔗
🔧 Code Smell 🚨 Major 🔵 Open Not assigned 5min effort 🗑️ cert, misra, unused

192 showDatePickerDialog(START_DATE);

193 }

194 }

195 public void showEndDatePickerDialog(View v) {

Remove this unused method parameter "v". ... 2 months ago ▾ L195 🔗
🔧 Code Smell 🚨 Major 🔵 Open Not assigned 5min effort 🗑️ cert, misra, unused

196 showDatePickerDialog(END_DATE);

197 }

198 }

199 public void showDatePickerDialog(int dataType) {

200 DatePickerFragment newFragment = new DatePickerFragment();

201 Bundle bundle = new Bundle();

202 bundle.putInt(DATE_TYPE_NAME, dataType);

203 newFragment.setArguments(bundle);

204 newFragment.show(getSupportFragmentManager(), "datePicker");

205 }

206 }

207 }

208 }

1474

41

Cyclomatic Complexity - app

←→↺🏠

localhost:9000/component_measures?id=unizar-vv-notas&metric=complexity&view=list

⋮☆

🔍Buscar

⬇️🌟➡️☰

sonarqube

ProjectsIssuesRulesQuality ProfilesQuality Gates

🔍Search for projects and files...Log in

📁app🔗master+

May 6, 2019, 4:21 PMVersion unspecified

OverviewIssuesSecurity Reports▼MeasuresCodeActivity

Project Overview

➤Reliability🔗

➤Security🔗

➤Maintainability🔗

➤Coverage

➤Duplications

▼Size

New Lines0

Lines of Code823

Lines1,446

Statements325

Functions97

Classes17

Files15

Comment Lines256

Comments (%)23.7%

▼Complexity🔗Cyclomatic Complex...

Cyclomatic Complexity132

Cognitive Complexity41

▼Issues

New Issues0

Issues50

Open Issues50

Reopened Issues0

Confirmed Issues0

app

View asList

⬆️⬇️to select files

⬅️➡️to navigate

15 files

Cyclomatic Complexity132📈

New code: since previous version

📄src/main/java/es/unizar/eina/notepadv3/Notepadv3.java32

📄src/main/java/es/unizar/eina/notepadv3/NoteEdit.java25

📄src/main/java/es/unizar/eina/notepadv3/NotesDbAdapter.java25

📄src/main/java/es/unizar/eina/notepadv3/test/Test_System.java13

📄src/main/java/es/unizar/eina/notepadv3/test/Test_NotesDbAdapter.java11

📄src/main/java/es/unizar/eina/notepadv3/dates/DateHelper.java7

📄src/main/java/es/unizar/eina/notepadv3/dates/AbstractDateImplementor.java5

📄src/main/java/es/unizar/eina/send/MailImplementor.java4

📄src/main/java/es/unizar/eina/send/SMSImplementor.java4

📄src/main/java/es/unizar/eina/send/SendAbstractionImpl.java3

📄src/main/java/es/unizar/eina/notepadv3/dates/DateMockImplementor.java2

📄src/main/java/es/unizar/eina/notepadv3/dates/DateRealImplementor.java1

📄src/main/java/es/unizar/eina/notepadv3/dates/DateImplementor.java0

📄src/main/java/es/unizar/eina/send/SendAbstraction.java0

📄src/main/java/es/unizar/eina/send/SendImplementor.java0

15 of 15 shown

🔔

Embedded database should be used for evaluation purposes only

The embedded database will not scale, it will not support upgrading to newer versions of SonarQube, and there is no support for migrating your data out of it into a different database engine.

Java Modeling Language (JML)

□ Lenguaje para especificación formal

❖ Utiliza la lógica de Hoare

➤ Precondiciones

➤ Poscondiciones

➤ Invariantes

❖ Las especificaciones se escriben como comentarios en Java

❖ <https://www.cs.ucf.edu/~leavens/JML/index.shtml>

Sintaxis

❑ Comentarios de Java

- ❖ `//@ <especificación JML>`
- ❖ `/*@ <especificación JML> @*/`

❑ Palabras reservadas

❖ **requires**

- Precondición

❖ **ensures**

- Poscondición

❖ **signals, signals_only**

- Poscondiciones relativas a lanzamiento de excepciones

Sintaxis

❏ Palabras reservadas

❖ **maintaining** o **loop_invariant**

➤ Definen el invariante de un bucle

❖ **decreases**

➤ Define la función de cota de un bucle

❖ **invariant**

➤ Define un invariante de una clase

Sintaxis

□ Expresiones

❖ **\result**

- Representa el valor devuelto por un método

❖ **\old(<expresión>)**

- Representa el valor inicial de <expresión> al comenzar la ejecución del método

❖ Cuantificadores:

- (<cuantificador> <declaración>; <dominio>; <cuerpo>)
- **\forall, \exists, \sum, \product, \min, \max, \num_of**

Ejemplo JML

```
public class A {  
  
    //@ ensures \result == true;  
    public boolean m() {  
        return true;  
    }  
}
```

Ejemplo con lógica de Hoare

```
public class Multiply {  
    /* Pre:  $n \geq 0$   
     * Post:  $\text{multiply}(m, n) = m * n$  */  
    int multiply(int m, int n) {  
        int product = 0;  
        int i = 0;  
        while (i < n) {  
            /* Inv:  $\text{product} = m * i \wedge 0 \leq i \leq n \wedge n > 0$  */  
            * Cota:  $n - i$  */  
            product = add(product, m);  
            i = i + 1;  
        }  
        return product;  
    }  
  
    /* Post:  $\text{add}(m, n) = m + n$  */  
    int add(int n, int m) {  
        return n + m;  
    }  
}
```

Ejemplo JML

```
public class Multiply {  
    //@ requires n >= 0;  
    //@ ensures \result == m*n;  
    int multiply(int m, int n) {  
        int product = 0;  
        int i = 0;  
  
        //@ maintaining product==m*i && i>=0 && i<=n && n>0;  
        //@ decreases n - i;  
        while (i < n) {  
            product = add(product, m);  
            i = i + 1;  
        }  
        return product;  
    }  
  
    //@ ensures \result == n + m;  
    int add(int n, int m) {  
        return n + m;  
    }  
}
```

Ejemplo JML

```
public class CopyArray {  
    //@ requires a.length == b.length;  
    //@ requires 0 <= iBegin && iBegin < a.length  
        && 0 <= iEnd && iEnd < a.length && iBegin <= iEnd;  
    //@ ensures (\forall int i; iBegin <= i && i < iEnd;  
        a[i] == b[i]);  
    public static void CopyArray(int[] b, int iBegin, int iEnd,  
        int[] a) {  
        int k = iBegin;  
        //@ maintaining iBegin <= k && k <= iEnd;  
        //@ maintaining (\forall int i; iBegin <= i && i < k;  
            a[i] == b[i]);  
        //@ decreases iEnd - k;  
        while (iEnd - k > 0) {  
            a[k] = b[k];  
            k = k + 1;  
        }  
    }  
}
```

Fuente: OpenJML, 2018. <https://www.rise4fun.com/OpenJMLES/CopyArray>

Métodos formales

□ TLA⁺

- ❖ Lenguaje para especificación formal
- ❖ Énfasis en sistemas concurrentes y distribuidos
- ❖ Válido también para sistemas grandes y complejos
- ❖ Se está implantando industrialmente
 - Intel
 - Especificación formal de *hardware*, desde 2002
 - Amazon
 - Amazon Web Services, desde 2011
 - Microsoft
 - Azure, desde 2015
 - ESA
 - Sistema operativo *OpenComRTOS*

Práctica 5.^a

2.^a parte. Análisis estático de código

□ Sección 3.1 del enunciado de la práctica 5:

- ❖ Ejecutad el inspector de IntelliJ
- ❖ Intentad corregir los problemas que se hayan encontrado
- ❖ Apuntad si cada una de las reglas os parece razonable o no

Inspección de código. *Code style issues*

❏ *Field may be 'final'*

❖ This inspection reports all fields, methods or classes, found in the specified inspection scope, that may have a final modifier added to their declarations.

❖ Solution

➤ `private final Item[] items;`

❖ Suppress

➤ `@SuppressWarnings("CanBeFinal")`

Inspección de código. *Declaration redundancy*

❏ *Unused declaration*

- ❖ This inspection reports classes, methods or fields in the specified inspection scope that are not used or not reachable from entry points. In editor, it also detects local variables that are declared but not used as well as private members that are never accessed for reading or not initialized in a class.
- ❖ Problem synopsis
 - Class has one instantiation but it is not reachable from entry points.
 - Class is not instantiated.
 - Method is not used
- ❖ Solution
 - Safe delete
 - Comment out
 - Add as Entry Point

Inspección de código. *Java 5*

❑ 'for' loop replaceable with 'foreach'

- ❖ Reports for loops which iterate over collections or arrays, and can be replaced with the foreach iteration syntax, available in Java 5 and newer.

❖ Síntoma

```
➤ for (int i = 0; i < items.length; i++) {  
    ... items[i] ...  
}
```

❖ Solución

```
➤ for (Item item : items) {  
    ... item ...  
}
```

❖ Suppress

```
➤ //noinspection ForLoopReplaceableByForEach  
➤ @SuppressWarnings("ForLoopReplaceableByForEach")
```

Fuente: JetBrains. IntelliJ IDEA 2016

Inspección de código. *Javadoc issues*

❏ Dangling Javadoc comment

❖ Reports dangling Javadoc comments. Javadoc comments are dangling if they don't belong to any class, method or field. For example a Javadoc comment in between method declarations that have their own Javadoc comments.

❖ Problem synopsis

```
➤ /**  
   *  
   */
```

❖ Solution

- Remove
- Replace with block comment

Inspección de código. *Numeric issues*

❏ Pointless arithmetic expression

- ❖ Reports pointless arithmetic expressions. Such expressions include adding or subtracting zero, multiplying by zero or one and division by one. Such expressions may be the result of automated refactorings not completely followed through to completion, and in any case are unlikely to be what the developer intended to do.
- ❖ Problem synopsis
 - 'item.getQuality() - item.getQuality()' can be replaced with '0' at line 56
- ❖ Solution
 - Simplify
- ❖ Suppress
 - *//noinspection PointlessArithmeticExpression*
 - **@SuppressWarnings("PointlessArithmeticExpression")**

Inspección de código. *Probable bugs*

□ StringEquality

- ❖ Reports any use of == or != to test for String equality, rather than the equals() method.
- ❖ Problem synopsis
 - String values are compared using '!=', not 'equals()'
- ❖ Solution
 - Replace with 'equals()'
- ❖ Suppress
 - *//noinspection StringEquality*
 - @SuppressWarnings("StringEquality")

Inspección de código. *Probable bugs*

- ❑ Reports any use of `==` or `!=` to test for String equality, rather than the `equals()` method.

- ❖ `item.getName() != "Aged Brie"`

- ❖ `!item.getName().equals("Aged Brie")`

- ❖ `item.getName() != "Sulfuras, Hand of Ragnaros"`

- ❖ `!item.getName().equals(
"Sulfuras, Hand of Ragnaros")`

Inspección de código. *Spelling*

□ SpellCheckingInspection

- ❖ Spellchecker inspection helps locate typos and misspelling in your code, comments and literals, and fix them in one click.

Inspección de código. *Abstraction Issues*

□ FeatureEnvy

- ❖ Reports the "Feature Envy" code smell. Feature envy is defined as occurring when a method calls methods on another class three or more times. Calls to library classes, parent classes, contained or containing classes are not counted for purposes of this inspection. Feature envy is often an indication that functionality is located in the wrong class.
- ❖ Problem synopsis
 - Class 'Item' accessed repeatedly in method 'updateQuality()' at line 13
- ❖ ¿Solución?
- ❖ Suppress
 - `@SuppressWarnings("FeatureEnvy")`

Inspección de código. *Abstraction Issues*

☐ MagicNumber

- ❖ Reports "magic numbers", which are numeric literals used without being named by a constant declaration. Magic numbers can result in code whose intention is unclear, and may result in errors if a magic number is changed in one code location but not another. The numbers 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 100, 1000, 0L, 1L, 2L, 0.0, 1.0, 0.0F and 1.0F are not reported by this inspection.
- ❖ Problem synopsis
 - Magic number '50' at line 24
 - ¿Solución?
 - Magic number '11' at line 26
 - ¿Solución?
- ❖ Suppress
 - @SuppressWarnings(" MagicNumber ")

Inspección de código. *Abstraction Issues*

❏ LocalVariableOfConcreteClass

- ❖ Reports any local variables whose type is declared to be a concrete class. Such declarations may represent a failure of abstraction, and may make testing more difficult. Declarations whose classes come from system or third-party libraries will not be reported by this inspection. catch block parameters of concrete exception type will also not be reported by this inspection.
- ❖ Problem synopsis
 - Local variable 'sulfuras' of concrete class 'Item'
 - ¿Solución?
 - Local variable 'inventory' of concrete class 'Inventory'
 - ¿Solución?
- ❖ Suppress
 - @SuppressWarnings("**LocalVariableOfConcreteClass**")

Inspección de código. *Abstraction Issues*

❏ ParameterOfConcreteClass

- ❖ Reports any method parameters whose type is declared to be a concrete class, rather than an interface. Such declarations may represent a failure of abstraction, and may make testing more difficult. Declarations whose classes come from system or third-party libraries will not be reported by this inspection.
- ❖ Problem synopsis
 - Parameter 'items' of concrete class 'Item[]'
 - ¿Solución?
 - Parameter 'items' of concrete class 'Item...'
 - ¿Solución?
- ❖ Suppress
 - `@SuppressWarnings("ParameterOfConcreteClass")`

Inspección de código. *Abstraction Issues*

❏ MethodReturnOfConcreteClass

- ❖ Reports any methods whose return type is declared to be a concrete class, rather than an interface. Such declarations may represent a failure of abstraction, and may make testing more difficult. Declarations whose classes come from system or third-party libraries will not be reported by this inspection.
- ❖ Problem synopsis
 - Method returns a concrete class 'Inventory' at line 18
 - ¿Solución?
- ❖ Suppress
 - `@SuppressWarnings("MethodReturnOfConcreteClass")`

Inspección de código. *Assignment issues*

❏ AssignmentToCollectionFieldFromParameter

- ❖ Reports any attempt to assign an array or Collection field from a method parameter. Since the array or Collection may have its contents modified by the calling method, this construct may result in an object having its state modified unexpectedly. While occasionally useful for performance reasons, this construct is inherently bug-prone.

- ❖ Problem synopsis

```
➤ class Inventory {  
    private final Item[] items;  
    public Inventory(Item[] items) {  
        this.items = items;  
    }  
    ...  
}
```

Fuente: JetBrains. IntelliJ IDEA 2016

Inspección de código. *Assignment issues*

```
public Inventory(final Item[] items) {
    super();
    // Copia superficial
    this.items = Arrays.copyOf(items, items.length);
}

public Inventory(final Item[] items) {
    super();
    // Copia profunda
    this.items = new Item[items.length];
    for(int i = 0; i < items.length; i++) {
        this.items[i] = new Item(items[i].getName(),
                                items[i].getSellIn(),
                                items[i].getQuality());
    }
}
```

Inspección de código. *Code Style Issues*

❏ ChainedMethodCall

❖ Reports method calls whose target is another method call.

❖ Ejemplo

➤ `item.getName().equals("Aged Brie")`

❖ Solución

➤ `String itemName = item.getName();
... itemName.equals("Aged Brie") ...`

➤ `item.isAgedBrie()`

Inspección de código. *Dataflow issues*

❏ LawOfDemeter

- ❖ Reports any Law of Demeter violations. See here http://en.wikipedia.org/wiki/Law_of_Demeter for an explanation what the Law of Demeter is.

Ley de Deméter

❑ Ley de Deméter o Principio del Menor Conocimiento

❖ «Solo hables con tus amigos más íntimos»

❑ En OO, se aplica a métodos:

Un método *m* de un determinado objeto **this** solo puede utilizar

- ❖ El propio objeto **this**
- ❖ Los parámetros de *m*
- ❖ Cualquier objeto creado o instanciado en el cuerpo de *m*
- ❖ Los atributos de **this**
- ❖ Cualquier variable global en ámbito

Ley de Deméter

```
((JLabel)  
myFrame.getContentPane().getComponent(0))  
    .setText("Hello");
```

- ❑ Técnicamente, no se rompe la encapsulación
- ❑ Realmente, somos conscientes de toda la estructura del panel gráfico de esa aplicación

Ley de Deméter

❑ Solución

- ❑ Añadir un método a myFrame
- ❑ Añadir un método al resto de elementos del *frame* hasta llegar al JLabel.

```
myFrame.setWelcomeText("Hello");
```

- ❑ Disminuye el acoplamiento
- ❑ Aumenta el número de métodos ofrecidos por las clases

Inspección de código. *Control flow issues*

❏ NegatedIfElse

- ❖ Reports if statements which contain else branches and whose conditions are negated. Flipping the order of the if and else branches will usually increase the clarity of such statements.

```
if (!item.getName().equals("Aged Brie") &&  
    !item.getName().equals("Backstage passes ...")) {  
    A;  
} else {  
    B;  
}
```

```
if (item.getName().equals("Aged Brie") ||  
    item.getName().equals("Backstage passes ...")) {  
    B;  
} else {  
    A;  
}
```